

```

1  """
2  Starter code for the problem "Cart-pole swing-up with limited actuation".
3
4
5  Autonomous Systems Lab (ASL), Stanford University
6  """
7
8  from functools import partial
9
10 from animations import animate_cartpole
11
12 import cvxpy as cvx
13
14 import jax
15 import jax.numpy as jnp
16
17 import matplotlib.pyplot as plt
18
19 import numpy as np
20
21 from tqdm import tqdm
22
23
24 @partial(jax.jit, static_argnums=(0,))
25 @partial(jax.vmap, in_axes=(None, 0, 0))
26 def affinize(f, s, u):
27     """Affinize the function `f(s, u)` around `(s, u)`.
28
29     Arguments
30     -----
31     f : callable
32         A nonlinear function with call signature `f(s, u)`.
33     s : numpy.ndarray
34         The state (1-D).
35     u : numpy.ndarray
36         The control input (1-D).
37
38     Returns
39     -----
40     A : jax.numpy.ndarray
41         The Jacobian of `f` at `(s, u)`, with respect to `s`.
42     B : jax.numpy.ndarray
43         The Jacobian of `f` at `(s, u)`, with respect to `u`.
44     c : jax.numpy.ndarray
45         The offset term in the first-order Taylor expansion of `f` at `(s, u)`
46         that sums all vector terms strictly dependent on the nominal point
47         `(s, u)` alone.
48     """
49     # PART (b) #####
50     # INSTRUCTIONS: Use JAX to affinize `f` around `(s, u)` in two lines.
51     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
52     c = f(s, u) - A@s - B@u
53     # END PART (b) #####
54     return A, B, c
55
56
57 def solve_swingup_scp(f, s0, s_goal, N, P, Q, R, u_max, p, eps, max_iters):
58     """Solve the cart-pole swing-up problem via SCP.
59
60     Arguments
61     -----
62     f : callable
63         A function describing the discrete-time dynamics, such that
64         `s[k+1] = f(s[k], u[k])`.
65     s0 : numpy.ndarray
66         The initial state (1-D).
67     s_goal : numpy.ndarray
68         The goal state (1-D).

```

```

81 N : int
82     The time horizon of the LQR cost function.
83 P : numpy.ndarray
84     The terminal state cost matrix (2-D).
85 Q : numpy.ndarray
86     The state stage cost matrix (2-D).
87 R : numpy.ndarray
88     The control stage cost matrix (2-D).
89 u_max : float
90     The bound defining the control set [-u_max, u_max].
91 p : float
92     Trust region radius.
93 eps : float
94     Termination threshold for SCP.
95 max_iters : int
96     Maximum number of SCP iterations.
97
98 Returns
99 -----
100 s : numpy.ndarray
101     A 2-D array where s[k] is the open-loop state at time step k,
102     for k = 0, 1, ..., N-1
103 u : numpy.ndarray
104     A 2-D array where u[k] is the open-loop state at time step k,
105     for k = 0, 1, ..., N-1
106 J : numpy.ndarray
107     A 1-D array where J[i] is the SCP sub-problem cost after the i-th
108     iteration, for i = 0, 1, ..., (iteration when convergence occurred)
109 """
110 n = Q.shape[0] # state dimension
111 m = R.shape[0] # control dimension
112
113 # Initialize dynamically feasible nominal trajectories
114 u = np.zeros((N, m))
115 s = np.zeros((N + 1, n))
116 s[0] = s0
117 for k in range(N):
118     s[k + 1] = fd(s[k], u[k])
119
120 # Do SCP until convergence or maximum number of iterations is reached
121 converged = False
122 J = np.zeros(max_iters + 1)
123 J[0] = np.inf
124 for i in (prog_bar := tqdm(range(max_iters))):
125     s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, p)
126     dJ = np.abs(J[i + 1] - J[i])
127     prog_bar.set_postfix({"objective change": "{:.5f}".format(dJ)})
128     if dJ < eps:
129         converged = True
130         print("SCP converged after {} iterations.".format(i))
131         break
132 if not converged:
133     raise RuntimeError("SCP did not converge!")
134 J = J[1 : i + 1]
135 return s, u, J
136
137 def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, p):
138     """Solve a single SCP sub-problem for the cart-pole swing-up problem.
139
140     Arguments
141     -----
142     f : callable
143         A function describing the discrete-time dynamics, such that
144         s[k+1] = f(s[k], u[k]).
145     s0 : numpy.ndarray
146         The initial state (1-D).
147     s_goal : numpy.ndarray
148         The goal state (1-D).

```

```

163     s_prev : numpy.ndarray
164         The state trajectory around which the problem is convexified (2-D).
165     u_prev : numpy.ndarray
166         The control trajectory around which the problem is convexified (2-D).
167     N : int
168         The time horizon of the LQR cost function.
169     P : numpy.ndarray
170         The terminal state cost matrix (2-D).
171     Q : numpy.ndarray
172         The state stage cost matrix (2-D).
173     R : numpy.ndarray
174         The control stage cost matrix (2-D).
175     u_max : float
176         The bound defining the control set `[-u_max, u_max]`.
177     p : float
178         Trust region radius.
179
180     Returns
181     -----
182     s : numpy.ndarray
183         A 2-D array where `s[k]` is the open-loop state at time step `k`,
184         for `k = 0, 1, ..., N-1`
185     u : numpy.ndarray
186         A 2-D array where `u[k]` is the open-loop state at time step `k`,
187         for `k = 0, 1, ..., N-1`
188     J : float
189         The SCP sub-problem cost.
190
191     """
192     A, B, c = affinize(f, s_prev[:-1], u_prev)
193     A, B, c = np.array(A), np.array(B), np.array(c)
194     n = Q.shape[0]
195     m = R.shape[0]
196     s_cvx = cvx.Variable((N + 1, n))
197     u_cvx = cvx.Variable((N, m))
198
199     # PART (c) #####
200     # INSTRUCTIONS: Construct the convex SCP sub-problem.
201     objective = 0.0
202     constraints = []
203     constraints += [s_cvx[0] == s0]
204     constraints += [cvx.norm(s_cvx[k] - s_prev[k], 'inf') <= p for k in range(N+1)]
205     constraints += [cvx.norm(u_cvx[k] - u_prev[k], 'inf') <= p for k in range(N)]
206     constraints += [cvx.norm(u_cvx[k], 'inf') <= u_max for k in range(N)]
207     constraints += [s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k] for k in range(N)]
208     objective += cvx.quad_form(s_cvx[N] - s_goal, P)
209     objective += cvx.sum([cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx[k], R) for k in range(N)])
210     # END PART (c) #####
211
212     prob = cvx.Problem(cvx.Minimize(objective), constraints)
213     prob.solve()
214     if prob.status != "optimal":
215         raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
216     s = s_cvx.value
217     u = u_cvx.value
218     J = prob.objective.value
219     return s, u, J
220
221 def cartpole(s, u):
222     """Compute the cart-pole state derivative."""
223     mp = 1.0 # pendulum mass
224     mc = 4.0 # cart mass
225     L = 1.0 # pendulum length
226     g = 9.81 # gravitational acceleration
227
228     x, theta, dx, dtheta = s
229     sintheta, costheta = jnp.sin(theta), jnp.cos(theta)
230     h = mc + mp * (sintheta**2)
231     ds = jnp.array(

```

```

245     [
246         dx,
247         dθ,
248         (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
249         -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
250         / (h * L),
251     ]
252 ]
253 )
254
255 return ds
256
257
258 def discretize(f, dt):
259     """Discretize continuous-time dynamics `f` via Runge-Kutta integration."""
260
261     def integrator(s, u, dt=dt):
262         k1 = dt * f(s, u)
263         k2 = dt * f(s + k1 / 2, u)
264         k3 = dt * f(s + k2 / 2, u)
265         k4 = dt * f(s + k3, u)
266         return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6
267
268     return integrator
269
270
271 # Define constants
272 n = 4 # state dimension
273 m = 1 # control dimension
274 s_goal = np.array([0, np.pi, 0, 0]) # desired upright pendulum state
275 s0 = np.array([0, 0, 0, 0]) # initial downright pendulum state
276 dt = 0.1 # discrete time resolution
277 T = 10.0 # total simulation time
278 P = 1e3 * np.eye(n) # terminal state cost matrix
279 Q = np.diag([1e-2, 1.0, 1e-3, 1e-3]) # state cost matrix
280 R = 1e-3 * np.eye(m) # control cost matrix
281 p = 1.0 # trust region parameter
282 u_max = 8.0 # control effort bound
283 eps = 5e-1 # convergence tolerance
284 max_iters = 100 # maximum number of SCP iterations
285 animate = False # flag for animation
286
287 # Initialize the discrete-time dynamics
288 fd = jax.jit(discretize(cartpole, dt))
289
290 # Solve the swing-up problem with SCP
291 t = np.arange(0.0, T + dt, dt)
292 N = t.size - 1
293 s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, p, eps, max_iters)
294
295 # Simulate open-loop control
296 for k in range(N):
297     s[k + 1] = fd(s[k], u[k])
298
299 # Plot state and control trajectories
300 fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
301 plt.subplots_adjust(wspace=0.45)
302 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
303 labels_u = (r"$u(t)$",)
304 for i in range(n):
305     ax[i].plot(t, s[:, i])
306     ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
307     ax[i].set_xlabel(r"$t$")
308     ax[i].set_ylabel(labels_s[i])
309 for i in range(m):
310     ax[n + i].plot(t[:-1], u[:, i])
311     ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
312     ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
313     ax[n + i].set_xlabel(r"$t$")
314     ax[n + i].set_ylabel(labels_u[i])
315 plt.savefig("cartpole_swingup_constrained.png", bbox_inches="tight")

```

```
# Plot cost history over SCP iterations
fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
ax.semilogy(J)
ax.set_xlabel(r"SCP iteration  $i$ ")
ax.set_ylabel(r"SCP cost  $J(\bar{x}^{(i)}, \bar{u}^{(i)})$ ")
plt.savefig("cartpole_swingup_constrained_cost.png", bbox_inches="tight")
plt.show()

# Animate the solution
if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save("cartpole_swingup_constrained.mp4", writer="ffmpeg")
    plt.show()
```